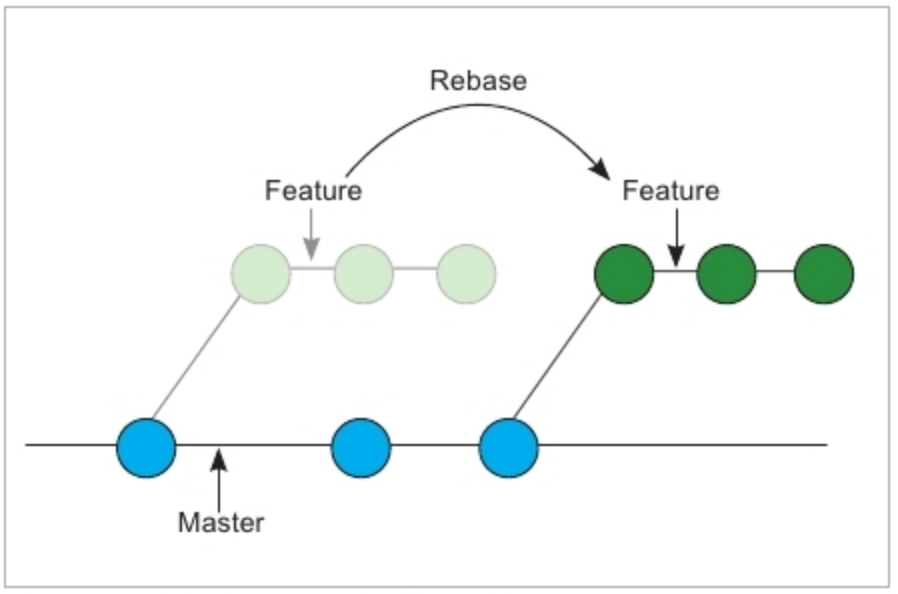

Figure 9: Branching structure after rebasing is done

```
(base) ekta@git-demo$ git remote -v
(base) ekta@git-demo$ git remote add origin https://github.com/ektanandwani/git-dem
o.git
(base) ekta@git-demo$ git remote -v
origin  https://github.com/ektanandwani/git-demo.git (fetch)
origin  https://github.com/ektanandwani/git-demo.git (push)
```

Figure 10: Adding a remote repository

commit messages, adding missing files in the commits, etc, but here we look at it in terms of integrating the work from two branches. This is called 'rebasing source onto target branch', which means taking the commit history from the source branch and reapplying target branch commits on top of it. It does not introduce *merge commit* unless there are conflicts with two branches. This leaves the target branch with a linear and clean history. This is done just like merging by checking out the target branch and rebasing the source onto the target. Some changes can be made in the *feature* and *master* branches to demonstrate rebasing, as shown in Figures 6 and 7.

```
(base) ekta@git-demo$ git push origin master
Username for 'https://github.com': ektanandwani
Password for 'https://ektanandwani@github.com':
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 4 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (15/15), 1.29 KiB | 263.00 KiB/s, done.
Total 15 (delta 1), reused 0 (delta 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/ektanandwani/git-demo.git
 * [new branch]      master -> master
```

Figure 11: Pushing the *master* branch to remote

```
(base) ekta@git-demo$ git pull origin master
From https://github.com/ektanandwani/git-demo
 * branch            master     -> FETCH_HEAD
Updating 19b3902..0c0da9e
Fast-forward
 README.md | 2 ++
 1 file changed, 2 insertions(+)
 create mode 100644 README.md
```

Figure 12: Pulling changes from origin to *master* branch

Rebasing is done using *git rebase <source-branch-name>* as shown in Figure 8.

As seen in the log, the *feature* branch contains a full history of the *master* branch as well as its own commits applied on top of *master* branch commits. The new branching structure is shown in Figure 9.

Working with a remote repository

A remote repository is a local repository hosted somewhere on the Internet. It is generally used to collaborate with other people, be it the team members or anyone who has access to the repository. *git remote -v* is used to see the remote server URLs along with the configured shorthand for the same. There should be none by default in a newly initialised Git repository, but for a cloned repository, the

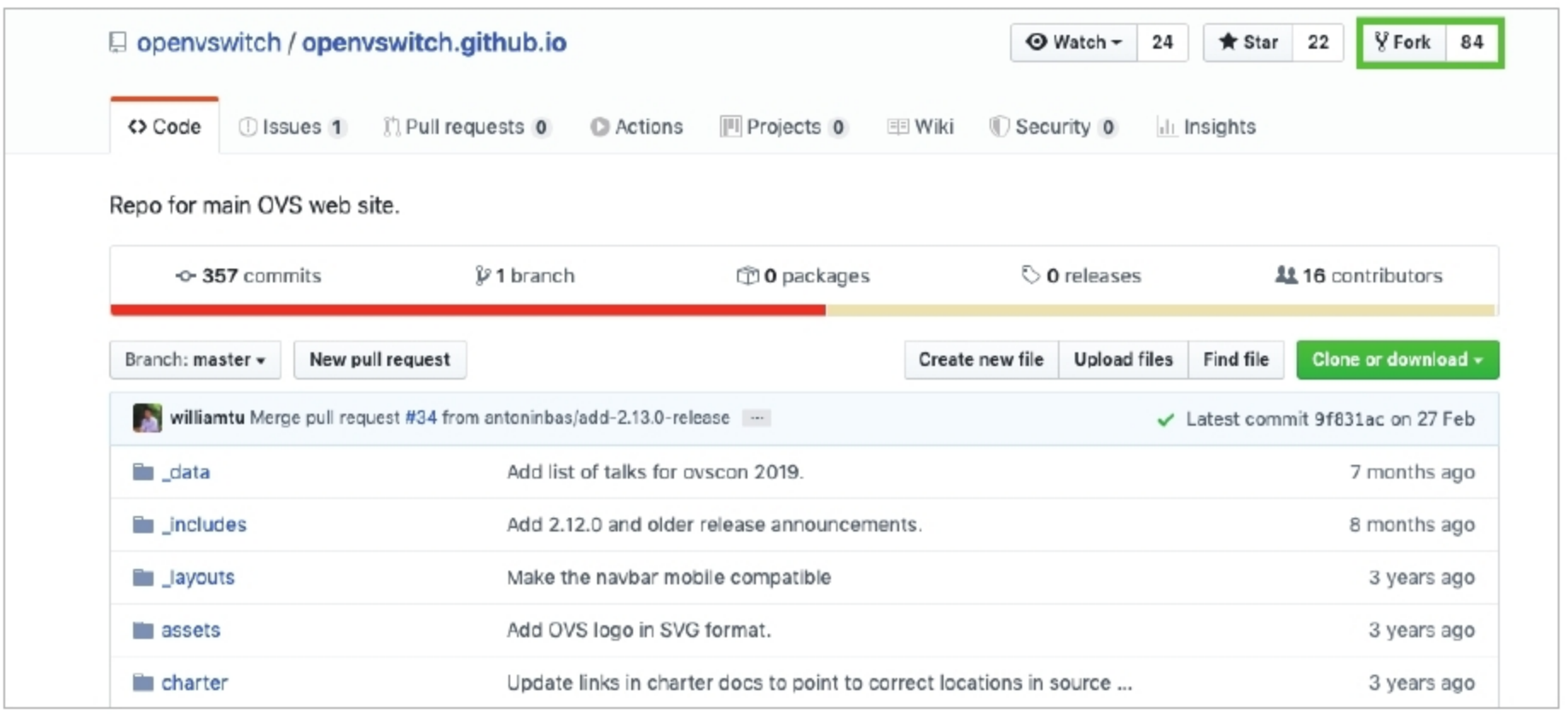

Figure 13: A remote open source repository used for forking